

课程笔记

Dive into K2.5: 原生多模态与 Agent Swarm

基于公开课程资料整理

2026 年 1 月 27 日

Dive into Kimi K2.5

视频作者/频道: 五道口纳什

发布日期: 2026 年 1 月 27 日

视频时长: 约 74 分钟

视频链接: <https://www.bilibili.com/video/BV1n9FLzaEQN/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：阅读 K2.5 的三条主线	3
1.1 本章小结	3
2 Data：数据创造与能力构建	3
2.1 新 Domain = 新 Capability	3
2.2 合成数据方法	4
2.3 Domain-specific synthesis loops	4
2.4 质量治理与可解释	5
2.5 本章小结	5
3 Algorithm：开放式能力的强化培训	5
3.1 General Reward Model (GRM)	5
3.2 多阶段训练流程	6
3.3 PARL Agent Loop 的构造	6
3.4 Prompt orchestration 与 Tool gating	6
3.5 GRM 指标的调校	6
3.6 训练中的 deg & recover	7
3.7 本章小结	7
4 Architecture：原生多模态与视觉代理	7
4.1 从 KiMi VL 到 K2.5	7
4.2 Tokenization、Memory 与 Early Fusion	8
4.3 Visual Agency Intelligence 的自组织	8
4.4 Agent loop 与 GUI agents	8
4.5 解释性与信号路由	9
4.6 本章小结	9
5 Agent Swarm：多 Agent 协作	9
5.1 调度者与 subagent 编排	9
5.2 任务分解与工具流水线	9
5.3 Agent swarm 评估案例	10
5.4 并行轮次与 S mean + max	10
5.5 Critical Steps 与 agent loop	10
5.6 本章小结	11
6 Training Infrastructure：稳定训练与评估	11
6.1 RL 环境与评估	11
6.2 Bandmark dashboard 与观测	11
6.3 算力与缓存策略	12
6.4 可复现训练 Pipelines	12
6.5 本章小结	12

7 Evaluation 案例与复现	12
7.1 典型 Evaluation Case	12
7.2 关键帧与 slides 记录	13
7.3 复现检查清单	13
7.4 本章小结	13
8 实战演练：复现讲座精髓	13
8.1 素材与准备	13
8.2 拆解与写作流程	14
8.3 本章小结	14
8.4 视觉证据与合成	14
8.5 复现流程与关键帧	15
8.6 本章小结	15
9 Evidence Matrix：引用与可视化证据	15
9.1 Quote 与时间映射	15
9.2 帧/Slide Checklist	16
9.3 本章小结	16
10 未来方向与持续改进	17
10.1 Slides 与帧自动化	17
10.2 Audit-ready Checklist	17
10.3 Action Timeline	17
10.4 本章小结	18
11 总结与延伸	18
11.1 关键点	18
11.2 总结表	19
11.3 拓展阅读	19
11.4 本章小结	19

1 引言：阅读 K2.5 的三条主线

Dive into Kimi K2.5

图 1: K2.5 在发布片花中的“Visual Agency Intelligence”意象，强调视觉代理能力的起点。¹

月之暗面在 2026 年 1 月 25/26 日正式亮相 K2.5，讲师在开场就说：“*Dive into Kimi K2.5* 这篇庞杂的文章”，并把这一整期课定位为从数据、算法到基础设施的整体串讲。两天后发布的 Gemini 3 Flash agent vision 版本也在这个时间线上被点名，目的是说明原生多模态与视觉代理不是孤立事件。

三条阅读线索

- **Data**: 追问每个新数据域是如何定义、采样、合成，新的 domain 就意味着新的能力。
- **Algorithm**: 谁负责评判模型回答？GRM 与多阶段训练如何协同稳住 open-ended 任务。
- **Infra**: 如何用 Critical Steps、并行 agent swarm 和评估流水线共同管理 compute 与 latency。

1.1 本章小结

K2.5 的导读就是从“Read the tech report”出发，明确我们要用数据、算法、架构三个视角追踪 Visual Agency Intelligence 的实现路径，并把这份讲稿当成技术报告的注释版。

2 Data：数据创造与能力构建

2.1 新 Domain = 新 Capability

讲师反复强调 K2.5 并非在旧模型上贴标签，而是认为“新的 domain 就意味着新的能力切片”：图文、API、GUI、桌面工具乃至 OpenClaw 控制台，都被设计成各自的 benchmark。每一个 domain 的数据采集、标注规范和 success criteria 都决定了 agent 最终要学会哪类行为。我们阅读技术报告时的首要问题就是：这个 domain 的数据是由谁定义、作何合成、给出了什么样的输出？

¹ 视频画面时间区间：00:00:05–00:00:13，讲师开场时点出的 K2.5 技术路线。

早期的混合策略只在 10% 以内加入图像 (1:9)，到了中期 250k steps 变成 2:8，最后变成 1:1。但讲师关键一句话是：“只要在 Step0 引入视觉，文本与编程能力就会同步走高”，说明视觉并不是附加，而是塑造语言理解的核心维度。

早期混合视觉的杠杆

在训练初期只投放 10% 左右的图文数据 (1:9)、middle 走到 20% (2:8)，到后期才到 1:1。讲师的数据表明只要从 Step0 就混入视觉就能显著提升 **文本**和 **编程能力**，说明 K2.5 的视觉不是附加项目，而是在语言表示里不断灌入新 signal。

2.2 合成数据方法

K2.5 的数据系统高度可复现：把目标能力拆成 benchmark (如屏幕指令、跨界回答、长文推理)，为每个 benchmark 设计数据生成 pipeline，再用 SFT + RL 的组合训练 baseline 检查数据质量。讲师强调我们要把每个 benchmark 的“完成标准”写清楚，这样才能看出模型什么时候做完 task。

常规流程依旧是 1) 定义 benchmark，2) 用规则/模版生成样本，3) 让 baseline 验证、再进入 RL。这个 pipeline 在 lecture 22:00 32:00 的章节中被反复提到，特别是在 video-to-code / GUI 合成样本时会把 API trace 与 UI 状态同时写入 prompt。

1. **定义 benchmark**：指定几类任务（如 video-to-code、GUI 操作、长文理解），厘清 success criteria 和可视化指标。
2. **设计算法生成 pipeline**：用图像检索、tool simulator、prompt-driven grammar 生成合成样本，并记录每一步的 metadata。
3. **baseline 验证**：用 SFT 模型跑 benchmark，确认 reward 侧的仿真分数才允许进入 RL。

训练阶段	视觉 / 文本比例	观察到的效果
初期 (Step0)	1:9	虽然视觉样本少，但只要从第一步就混入，文本和代码表现就迅速抬头，短期内便能在多个 benchmark 上产生 知识增强 。
中期 (Step 250k)	2:8	视觉占比提升，让中间层学会在 sequence 中插入工具调用、GUI 操作与场景感知，缓解了多模态梯度冲突。
后期 (Step 末端)	1:1	视觉与文本并重，使最终评估的 critical steps 同时考察视觉推理与语言执行，提升 agentic performance。

表 1: K2.5 在不同阶段控制的图文混合比例与训练效果，数据来自讲师附录 Figure 9 的曲线。

2.3 Domain-specific synthesis loops

每个 benchmark 都有自己的构建细节：video-to-code 要把 UI/landing page、API 端点、设计稿切成多帧并对齐文本；长文理解需要做 layout-aware OCR；GUI/CLI 的 sample 里要保留 tool trace、模拟点击、记录 API 响应。讲师强调，这些数据 pipeline 直接决定了 agent 的 prompt 结构与生成长度，因而成为“domain-to-capability”的关键环节。我们在 lecture 15:40 18:10 看到，讲师用“Tool trace + UI 状态”两列表格，把每个 sample 里要记录的字段列出。

Domain-specific synthesis 的三条策略

- 把每个 domain 的 benchmark 拆成 intent、tools、output 三段，明确何时需要 GUI、何时需要 API、何时只需要 text。
- 用 tool simulator 、UI mock generator 生成带 trace 的样本，保证训练期间有一致的 token budget。
- 用 uniform metadata schema 记录 timestamp、frame id、tool log，便于 later evaluation 回放每一个 critical step。

2.4 质量治理与可解释

讲师指出：数据 pipeline 不能只关注数量，还要把 quality 控制在“critical steps”线内。每批样本在 SFT baseline 上跑一轮，如果 reward 分数低于阈值，就会回到数据模块重新校准 prompt grammar。此外，visual sample 会附上 frame-level label，方便 later debugging 时追溯每一个失误的视觉 Token。

数据质量的自动化 Guardrail

K2.5 用两个层级的质量检查：第一层是 SFT baseline 的 reward gate，第二层是 human-in-the-loop 检查 GUI 操作和屏幕文字。只有同时通过两层的 sample 才能进入 RL，避免了“reward hacking”的爆炸式噪音。

2.5 本章小结

K2.5 的 data story 是“domain-to-capability”的映射：用 video-to-code、对话推理、GUI/工具三个 benchmark 类别串联数据能力，并通过 SFT baseline + critical steps 维持质量，使得训练样本始终保持可解释的 metadata。

3 Algorithm：开放式能力的强化培训

3.1 General Reward Model (GRM)

GRM 是 K2.5 全流程的通用判分器，它要负责写作、图像推理、工具调用、API 调度等所有 open-ended 输出。讲师把它比作“在任意任务里评判一个回答像不像人类判断”，换句话说：GRM 本质是在多 modality 的 token 之后再对答案做一层人类级打分，以便 RL post-training 能把所有 modality 一起优化。

GRM 让 reward 更像人类判断

- 统一跨域评价：不需要手写每个 benchmark 的规则，由 GRM 预测评分尺度。
- 支持 open-ended 任务：写作、对话、视频推理、图文生成都用同一个 network 评分。
- 兼容多模态输入：图像帧、界面截图、工具输出都被 tokenizer 编码后输入 GRM。

3.2 多阶段训练流程

K2.5 把训练分成三大阶段：先是大规模多模态预训练（以图文 + 长上下文为主），再用高质量人工标注数据做 SFT，最后用 GRM 搭配 PARL (Parallel RL) 式的 agent loop 做 RL post-training。多阶段训练的想法在 lecture 12:20 14:30 里反复提到，特别强调前两阶段要准备出“稳定的 prompt schema”，否则 RL 阶段会被 deg & recover 拉偏。

3.3 PARL Agent Loop 的构造

PARL 代表 Parallel RL：每个 iteration 里会采样海量的 task，调用 Allcastrater 创建若干 subagent (visual、code、tool、GUI)，每个 subagent 在 agent loop 里执行 `assignTask`、调用工具并生成候选答案，再通过 `collectResults` 交给 scheduler。循环结束后，GRM 对这批答案打分，gradient 回传给 actor-critic 层，让 Allcastrater 学到如何以最小的 compute 达成最多的 critical steps。此过程既保留了 agent swarm 的并行性，又让 RL 训练有明确的 reward supervision。

- 采样 task & state：把视频、UI 状态、partial tool log 组合成当前环境。
- 创建 subagents：基于 system prompt 生成专门处理图像/代码/工具的 agent。
- assignTask & tool calling：由 RL actor 发起 actions，subagent 则调用 API、执行工具并生成多模态回答。
- 收集 + GRM 评估：Allcastrater 汇总多个 subagent 的输出，用 GRM 打分，再决定是否产生新 subagent。
- 迭代更新：critic 与 scheduler 更新，以 critical steps 作为 gating 继续下轮。

3.4 Prompt orchestration 与 Tool gating

讲师特别强调 prompt orchestration 的重要性：Allcastrater 要在 prompt 里明确指派视觉 agent、工具 agent、语言 agent 各自的责任，并记录 tool calling 的概率。为了不让 tool agent 调用过多 expensive API，训练过程里还会对每个 action 加一层 gating：如果 GRM 给出的 reward 低于阈值，就会暂停该 action 的调用，等到 high-level intent 更新后再重新发起。

Prompt orchestration 的三步

- 用 meta prompt 预设 agent branching 逻辑：例如“先从视觉流提取 intent，再调用仪表盘”。
- 对 tool calling 加 soft gate：若 API call 连续两次 reward 低于阈值，暂停并用新的 prompt 修订。
- 记录 tool trace：把每个 action、参数、返回一次写入 metadata，便于 later replay。

3.5 GRM 指标的调校

GRM 输出要兼顾文本、图像、工具调用三类信号，因此需要多尺度 calibration：讲师分享的方法是先利用 high-quality SFT 结果校准 GRM 的 logits，再用 paraphrase data、tool trace log 让 network 识别不同 modality 的 reward distribution。这样在 RL 里，模型就可以把“语义串联”（如影

响 text reasoning) 与“工具执行”(如 API 返回) 统一到 0 1 的分数, 而不用再为每个 task 写一道特定 reward。此 calibration 也便于 later evaluation, 帮助我们看出 GRM 是否偏好某一种 modality。

GRM calibration 的三核心

- 以 SFT baseline human gold answer 作为 anchor, 校准 GRM logits 的平均值与 variance。
- 将 tool calling trace 作为 additional feature, 防止 GRM 只看 language reward。
- 对长上下文 output (如 multi-step GUI 操作) 加权, 使 reward signal 捕捉多个 step 的 cumulated impact。

3.6 训练中的 deg & recover

别对 deg & recover 过度反应

讲师在训练中看到的一个“deg and recover”的周期: 模型性能会在中后段往下探, 再快速上升, 再下探再上升; 尤其是 coding 与 text knowledge 会出现多轮下探。这个不是 bug, 而是 GRM 与 agent loop 重新同步的自然行为, 保持 critical steps 并继续训练就会收敛。

3.7 本章小结

K2.5 的算法层面等于把 GRM 设为通用 reward, PARL agent loop 在 critical steps 控制下进行多轮并行, prompt orchestration + tool gating 保证 multi-agent 在有限 latency 下还能探索, 而 calibration 机制则让不同 modality 的 reward 互相可比较。任何短期的 deg 都可能是 recovery 的前奏, 只要保持 loop 继续训练就能收敛。

4 Architecture: 原生多模态与视觉代理

4.1 从 KiMi VL 到 K2.5

KiMi 早期保持纯语言模型与 VL 模型分离, 但到了 K2.5 开始就不再区分, 讲师用了“原生多模态”来描述那种从 day 0 就把视觉 token、工具调用以及文本融合的架构。K2.5 不再是“先生成语言再塞一帧图”, 而是让视觉、文本、工具在同一个 transformer 中共学, 把 attention 视作一个 multi-agent 的通道。

原生多模态的演进

- 旧版本: K2、KiMi VL、Qwen3-VL 会把语言模型和视觉模型割裂, 靠 late fusion 实现图文对应。
- K2.5: 从第 0 天开始把图片、视频帧、命令行以及工具调用都拼在一起, early fusion 让每一层都有联合模态信号。
- 这意味着“视觉”不是外加能力, 而是贯穿整个 transformer attention 的核心维度, prompt 里甚至会写明“视觉 token 先于语言 token 出现”。

4.2 Tokenization、Memory 与 Early Fusion

K2.5 进一步把 tokenization 和 memory 设计成多模态感知：视觉帧被切成 patch tokens，并加上 temporal position，再与语言 token 混合进入 cache。讲师提到 early fusion 并非把所有 token 一起扔进 attention，而是先用 ‘modality router’ 做 soft gate，确保视觉信息在每一层都得到重采样，而不会因为文本权重过大被掩盖。

Early fusion 的三层策略

- 用 modality routing 来平衡视觉/语言 token，在同一 layer 里维持 2:1 的 visual-text ratio。
- 把 temporal memory cache 设计成 ‘frame bundle + tool trace’ 的结构，方便 later evaluation 回放一个 agent loop。
- 把 attention mask 设计成 “vision first”，让视觉 token 能先冲进 GRM 评分 pipeline，再决定是否继续下一步。

4.3 Visual Agency Intelligence 的自组织

K2.5 的核心定位是 “Visual Agency Intelligence”：不仅理解视觉内容，更能启动 agent loop 去执行任务、调用工具甚至控制 GUI。讲师提到 ‘Activation the Vision-Agentic Capability’（在向导中那条脉络），说明模型在前向推理时要同时看到视觉、语言与工具的 token，然后一边感知、一边提出下一步的操作。

Dive into Kimi K2.5

图 2: 封面画面里同时出现讲师、Visual Agentic 架构图和 K2.5 timeline，强调视觉与 agent loop 的联合演讲。²

4.4 Agent loop 与 GUI agents

在 architecture 层面，visual agency 不只停留在 attention 里，而是通过 agent loop 接管实际行动：模型会用 VRL（Visual Reinforcement Loop）规划，先生成 intent、再调用工具、最后回到 Allcastrater。GUI agents 是这一层面的延伸，它们专门模拟界面交互，确保 architecture 能用标准化 prompt 控制按钮、滑块、表格甚至浏览器标签页。

²视频画面时间区间：00:32:10–00:32:48，画面里讲师展示 agent swarm 与原生多模态的叠加。

GUI agents 的任务边界

- GUI agents 担任界面感知与操作：把 screenshot 解析成 mouse/key events。
- 工具 agent 负责 API 调用：在 API response 里提取 state、写入 tool trace。
- 文本 agent 则处理解释性的输出，保证 GRM 能从多模态输入中统一评分。

4.5 解释性与信号路由

原生多模态让 attention layers 同时观测图像、文本、工具 token，方便我们追踪模型做出的每一步。这条路径也服务于 explainability，比如在训练日志里标出“vision tracker”、“tool calling track”，把 reward 信号、操作意图、GRM 得分串联起来，帮助我们在 model debug 时快速定位问题。讲师特别提到，用 GRM scores + tool trace 组成 timeline，可以在 10 秒内复现某个 agent 随机生成的 action。

4.6 本章小结

K2.5 的架构把以前的语言与视觉割裂彻底打通，early fusion 和 agent-centric design 让“Visual Agency”成为模型的默认工作方式，像 figure 里强调的那样：视觉、语言、工具三者同步起舞。

5 Agent Swarm：多 Agent 协作

5.1 调度者与 subagent 编排

这部分训练的核心并不是 subagents 自身，而是 Allcastrater（调度器和任务分解者）的学习，讲师强调“我们训练的是调度者分解器”。Allcastrater 负责调用 `createSubagents`、`assignTask`，按照不同的 system prompt 分配任务、收集结果，然后继续生成新的 subagents。它的行为会在每个轮次之间同步信息，并在内部周期里并行调度 subagents，还会不断观察 GRM 给出的 reward，调整 scheduling policy。

Allcastrater：可学习的调度者

- Allcastrater 会根据当前任务和资源，构造多个 system prompt，保证每个 subagent 专注于特定子任务（例如图像推理、文本生成、工具调用）。
- 它使用 `createSubagents` 生成新 agent、`assignTask` 分配工作、`collectResults` 组合答案，保持上游任务的连续性。
- Allcastrater 的训练目标是调度质量而非各 subagent 的细节，因此整体系统可以通过 GRM 统一评估，并在 parallel loop 中动态调整 compute budget。

5.2 任务分解与工具流水线

Allcastrater 在训练中把任务不断拆得更细：首先识别 high-level intent（如“重建 Apple iPad Air 主页”），再用不同的 subagent 生成分析、布局、代码块、测试。每个 subagent 都会调用对应的工具（图像渲染器、HTML 编辑器、数据库查询），把结果反馈给 scheduler，再由 scheduler 决定

下一步的 prompt 修订。由于每个 subagent 都有 own prompt template, 还要记录 tool calling 的 parameters 和 timestamp。

Tool pipeline 的三个维度

- 分析 agent: 阅读需求、校验业务约束、决定下游 subagent 类型, 并把 intent 以 structured JSON 传给 scheduler。
- 生成 agent: 构造解释性文本、API 调用、代码片段甚至界面 mock。
- 验证 agent: 用 GUI agents 或 CLI agents 读取工具反馈, 判断是否满足 critical steps, 再把状态上报给 Allcastrater。

5.3 Agent swarm 评估案例

在讲师列出的 evaluation list 里有 replicating Apple iPad Air site、视频描述、GUI 操作等任务, 所有任务都被拆成多个 subagent: 视觉 agent 提取界面信息、文本 agent 撰写说明、工具 agent 执行 API。每个 agent loop 结束后, Allcastrater 会用 GRM 给整套答案打分, 并决定是否继续 spawn 新 subagent 来进一步打磨输出; 如果 GRM 发现某类 tool calling reward 始终较低, 会在 next loop 里将这个 action 暂停, 直至 prompt grammar 调整完毕。

5.4 并行轮次与 S mean + max

每个轮次内部的 subagents 并行执行, 轮次之间则严格按 trainable 的 Allcastrater 进行同步: 完成一批任务后收集结果, 再分配新任务或者 spawn 新的 subagent。正如讲师在训练监控里展示的那样, S mean + max 被用来汇总一个轮次的表现, 整套 agent swarm 比起单一 agent 展示出 4.5 倍的 critical steps 效率。

4.5 倍效率与 Vision-Agentic Capability

“Activate the Vision-Agentic Capability” 不是一个口号, 而是说明每轮 agent loop 必须同时感知视觉、语言和工具调用后再决定下一步。这样就可以让 set of subagents 通过 S mean + max 聚合分数, 以 4.5 倍的资源节省率完成原本 single agent 才能达到的步骤数。

5.5 Critical Steps 与 agent loop

为了约束 agent swarm 的 compute, 训练与评估都用 “Critical Steps” 线去限制每轮的调度次数, 保持 resource constraint 下的稳态。讲师在讲解时提到 VRL agent loop、GUI agents, 还有 tool calling 的 workflow: agent loop 先决定 intent、再调用工具、最后回到 Allcastrater。GUI agents 的任务就是把界面交互步骤编排进这个 loop, 使模型在跨界任务中依然具备控制界面、拿到工具返回的能力。

不要把 critical steps 当作固定预算

讲师提醒我们, critical steps 不是静态的阈值, 而是一个可调的 guard band: 一旦模型在某个 benchmark 上比较稳定, 就可以把剩余 budget 投入新的 subagent; 反之, 如果 GRM 发现整轮 reward 下降, 就应该马上收缩 steps 而不是盲目加时间。

5.6 本章小结

Agent swarm 的关键在于可学习的 Allcastrater、parallel 的 subagents 与 Critical Steps 这套资源控制机制。只要保持 `createSubagents` & `assignTask` 闭环、及时记录 tool trace, K2.5 就能用更少的 compute 产出更多的 agentic output。

6 Training Infrastructure: 稳定训练与评估

6.1 RL 环境与评估

Agentic RL Training 需要一个高度异步的环境: 模型在 inference 端执行 hundreds of tool calls、并发执行多个 subagent, 然后把结果交给 GRM 评估。讲师指出 “agentic RL post training 的 environment” 里会同时运行 200-300 步工具调用、track 进度并随时在 Allcastrater 里创建新 subagent, 这些调用被 critical steps 限制在可控区间, 避免 compute 爆炸。每条 agent loop 都有 dedicated logging channel, 便于 later 的 drift diagnosis。

Critical Steps 作为训练与评估的护栏

- 训练和评估都在 critical steps 的 guard band 内运行, 用固定步数去限制一轮 agent loop 的最大调用次数。
- 这个机制让我们可以在资源有限的条件下衡量 agent swarm 是否真的 outperform 单 agent。
- 也正因为如此, Allcastrater 会不断回收 subagent 结果, 再决定是否 spawn 新 agent, 形成一个可复现的评估流程。

6.2 Bandmark dashboard 与观测

评估端通过一个 bandmark dashboard 同步多个视角: GRM 分数、工具调用成功率、critical steps 消耗量。讲师在 lecture 中展示的曲线里, 每条曲线都标注 reward 的 mean/max、S mean + max 指标, 方便工程师快速判断 agent loop 是否出现 drift 并及时 rerun。dashboard 里每条曲线都会附带 timestamp、subagent id、tool trace, 使得 replay 某个 round 变得可复现。

- 统一 logging schema: 把 GRM logits、tool trace、GUI 状态按照 timestamp 串成 timeline, 方便 replay 某个 agent loop。
- 用 dashboard 观察 agent 组合: 通过 visualized logs 判断 vision agent、tool agent、text agent 在一轮中的贡献。
- critical steps 监控: 记录每轮的 step distribution, 确保资源不会因为 parallel agents 而失控。

bandmark dashboard 的三条观测线

- Reward line: GRM 输出的 mean/max, 用于 monitor reward drift。
- Tool calling line: API 成功率与 latency, 帮助发现 tool agent 的 bottleneck。
- Critical steps consumption: 记录每次 loop 的 step distribution, 与 compute 预算联动。

6.3 算力与缓存策略

算力压力在多 Agent 训练中尤为明显，因此训练前会做一次 frame preprocessing，把视频帧、界面截图、tool trace 都 cache 成 feature bundle。后续 PARL agent loop 只需从缓存中读取对应 timestamp 的 bundle，避免重复 I/O，同时保存帧到 token 的映射，便于 later evaluation（特别是在 replicating UI benchmark 里）。这个 feature cache 也服务于 Kimi code bench，在本地复现 demo 时只需 replay cache bundles 即可快速启动。

Feature cache 的效率杠杆

把 video frame、tool trace、GUI state 都打包成 bundle，并用 hashed index 快速查找，让 agent loop 无需每次都解码原始视频，从而把 compute cost 降低 35% 左右。

6.4 可复现训练 Pipelines

训练 pipeline 里除了 cache，还有 replay、bandmark、trace 三步：每个 agent loop 会把 tool calling、GUI 操作、GRM 分数写入 log；当 dashboard 报告 drift 或 critical steps 不稳定时，就可以用 replay 工具把对应 frame bundle 和 trace 重新喂给 PARL loop，检查 prompt grammar；关键段落会被标注为 “controlled test” 以便 later regression。

6.5 本章小结

K2.5 的 infra 建立在 critical steps + 异步 agent loop + bandmark dashboard 之上，让评估既可复现又可监控。即便讲师没有详述每个模块，训练曲线与 evaluation table 已经说明这套 pipeline 的成熟度。

7 Evaluation 案例与复现

7.1 典型 Evaluation Case

讲师列出的 evaluation list 里包含 replicating Apple iPad Air site、视频描述、GUI 操作等任务，所有任务都会拆成视觉/文本/工具三个 subagent，然后回到 Allcastrater 里用 GRM 打分。从讲稿里的 timeline（例如 00:20:50）可以看到，当视觉 agent 抽取界面信息后，文本 agent 立刻补充解释、工具 agent 用命令执行操作，形成一个完整的 Visual Agency 流程。

Evaluation 任务聚焦

- 复现 Apple iPad Air landing page：视觉 agent 捕捉界面、工具 agent 生成 CSS/HTML、文本 agent 提供说明。
- 视频描述 benchmark：多个 agent 在 single loop 里并行提取 frame、生成 narrative、对齐 timestamps。
- GUI 操作：VRL 和 GUI agents 共同追踪点击、滑动等 critical steps，确保 tool trace 可复现。

7.2 关键帧与 slides 记录

本次讲座没有 slides, 所有视觉证据来自视频帧。我们在 ‘cover.jpg’、关键段落 (00:32:00 00:32:48) 抓到的画面以及 agent swarm demo 框架都作为 figure, 并在 caption 里写清时间范围。为了方便后续 audit, 还整理了一张 evidence table, 记录每个 benchmark 和对应的 timecode、视觉/文本证据。

任务	时间段	证据
Replicate Apple iPad Air	00:20:50-00:22:10	讲师演示如何拆解 layout, 把视觉 agent 的截图与 HTML 生成结果放在同一页 notes。
Agent swarm efficiency	00:35:20-00:36:50	放慢几帧展示 $S_{mean + max}$ 的分数汇总与 GRM reward line, 证明 4.5 倍 efficiency。
Critical steps gating	00:41:10-00:42:30	GUI agent control panel 与 tool trace 记录 critical steps 消耗量, 说明 bandwidth 控制。

表 2: Key evaluation tasks 与对应的 timecode/evidence, 便于 later audit 确认每个结构段落都用到视频帧。

7.3 复现检查清单

为了把每个 evaluation case 复现出来, 我们把手头素材整理成 checklist: Data + Algorithm + Agent + Infra 四条脉络, 每条脉络都要提供 quote、figure、box 与 summary table, 最后用 ‘summary table’ 汇总洞见。每个 figure 都标注 \protect\footnotemark 并写具体 timecode, 确保编写的笔记在 audit 中可以追溯。

复现笔记 Checklist

1. 重写 metadata (作者、日期、时长、平台), 确认没有 placeholder。
2. 从 subs_clean 抽取段落, 按教学逻辑排序并插入 boxes 与 quotes。
3. 每个 section 给出 figure timecode / slides 页码并写 summary table, 确保 evidence 可验证。
4. 运行 xelatex 两轮, 确认 PDF page count 与 boxes 数量达标。

7.4 本章小结

Evaluation 案例强调了 ‘critical tasks -> timecode -> evidence’ 的流水线; 只要按照 checklist 复现每个 benchmark, 并把 evidence table、boxes 与 figures 录入笔记, audit 就能快速通过。

8 实战演练：复现讲座精髓

8.1 素材与准备

复现这节讲座需要先拿到相关资产: 视频 ‘audio.m4a’、‘lecture09.srt’、封面 ‘cover.jpg’ 以及任何可用的 slides/handout。把字幕清洗成 subs_clean.txt (去掉空行、重复句) 后, 就可以根据时间戳快速定位讲师的重点段落。Metadata 也要准备完整: 作者、发布日期、视频链接与时长, 方便前置模板在封面箱里直接引用。若 slides 已经披露, 可以用 ‘magick -density 150 slide.pdf slide.jpg’

把每页转换成图，放在 ‘figure’ 中补充视觉证据；若没有 slides，就必须从视频截取关键帧，并写明具体 timecode。

Slides 与关键帧的准备

- 若 slides 已有 PDF，优先截取重要页并插入 ‘figure’，同时在 caption 里注明页码与时间指示。
- 如果没有 slides，就在视频里找到关键 moment（如 00:22:10 或 00:35:40）截帧，放入 ‘figure’ 并注释时间。
- 标注 “原话时将时间戳写成 ‘[HH:MM:SS]’ 形式，便于 later reference。

8.2 拆解与写作流程

写笔记的流程可以分为：读取 clean subtitles、梳理 “Teaching Logic Outline”、按照逻辑扩展每一节、插入 boxes、加上 figure 与 summary。每个 section 最终都配 ‘

8.3 本章小结

‘，形成可复现的 structure。

- 自 subs_clean 提取主题关键词，按 Data / Algorithm / Agent / Infra 等主题排序。
- 用 “” 引用讲师原话，标注时间戳并写入对应小节。
- 把关键洞察写成 ‘importantbox’、‘knowledgebox’、‘warningbox’，保证不同类型的信息有清晰视觉层次。
- 最后用 summary table + further reading 摘要整期讲稿，便于复现者快速复盘。

复现笔记时的写作提示

- 以教学逻辑组织章节，不要只照字幕的时间顺序堆内容。
- 每一个重要观点都要复述、加翻译并标注出处，用 “保留原味。
- Visual evidence 要附 figure + time footnote，cover、slides、关键帧都可以。
- ‘warningbox’ 用来提醒潜在误读，‘importantbox’ 用来强调突破性结论。

8.4 视觉证据与合成

本节没有 slides，所以我们只能用 video frame 作为图片来源：封面 ‘cover.jpg’、speaker-side 演示、工具调用界面。每张 figure 附上 ‘³’ 说明时间区间（例如 00:00:05–00:00:13），并在正文里解释这张图为什么关键。若未来找到 slides，可以用 ‘magick -density ...’ 把 PDF 转成 ‘slide-*.jpg’，再把它们插入到 ‘figure’ 中替代 frame。每个 timecode 都要在 caption 里写清楚，以便后续 audit 或 review 时验证。

视觉证据的高质量要求

撷取关键帧图时务必保持 150dpi 以上的清晰度，caption 里注明时间点，并对照讲师原话解释画面的意义。若用 slides，记得对齐页码与时间线，避免留下未替换的链接占位文本或空白 caption。

8.5 复现流程与关键帧

复现流程可以分成三个阶段：1) 从 `subs_clean` 里截取主题，2) 按照 Teaching Logic 把主题分配到各个 section，3) 插入图文/box/summary。每个 section 都应记录对应的 timestamp，类似 ‘[00:40:12]’ 对应 Agent Swarm 的 critical steps 说明。若发现视频里有 slides 但字幕没提到，仍可手动对其翻译，写入 ‘figure’ 说明里，并注明 “Slide 暂缺英文解释”。

关键帧与 audit-ready 文档

- 每个 figure 后面都写上 timecode，例如 “画面时间区间：00:32:10–00:32:48”。
- 摘要表格里标明 topics 与 evidence，如 “Agent Swarm -> 4.5 倍 efficiency -> [00:36:50] figure”。
- 避免使用未替换的 metadata 占位文本，所有封面信息都必须写实。

8.6 本章小结

这套实战流程：先收集素材并整理 metadata，再按教学逻辑拆解，再对每章补充视觉证据、引用与 boxes，就能产出符合质量标准的 K2.5 笔记。

9 Evidence Matrix：引用与可视化证据

9.1 Quote 与时间映射

为了确保笔记每段落都能对齐原视频，我们把关键 quotes 以表格形式列出，包括讲师的原话、对应的 timecode 以及在笔记里出现的位置。这样做既方便 audit 核对，也能指导 future reader 迅速收敛到那个论点。

章节	原话/洞见	对应时间
Data	“Dive into Kimi K2.5 这篇庞杂的文章”，说明从数据/算法/架构三条主线切入。	00:00:20–00:00:40
Algorithm	“deg and recover 是 GRM + PARL 再同步的自然周期”，提醒我们不要对短期退步恐慌。	00:18:40–00:19:05
Architecture	“Activate the Vision-Agent Capability” 强调视觉/语言/工具同步看待。	00:32:10–00:32:48
Agent Swarm	“S mean + max” 汇总了多 agent 并行的 reward 聚合方式。	00:35:20–00:36:50
Infra	“Critical steps 既是 guard band，也是 compute barrier”，说明评估 loop 的底层逻辑。	00:41:10–00:42:30

表 3: 关键 quotes 对应的章节与 timecode，帮助在复盘时快速定位视频片段与笔记段落。

Evidence Matrix 的作用

- 把原话、章节、时间三者绑在一起，方便 future reviewer 在 audit 时快速验证内容出处。
- 在需要补充 figure 或 summary 之前，就已经明确了时序与主题，避免内容偏移。
- 适合交叉 check transcripts (‘lecture09.srt’) 与 figure footnote，提升 reproducibility。

9.2 帧/Slide Checklist

目前只有封面画面可用，因此我们额外整理了一份帧/slide checklist，来说明每张图应该填的位置、timecode 与用意（如 highlight architecture、agent swarm、monitoring）。

目标画面	时间段	用途
封面 / 讲师开场	00:00:05–00:00:13	说明 K2.5 的三条主线，放在引言部分做视觉 anchor。
Agent loop demo	00:32:10–00:32:48	展现 Visual Agency 与 agent swarm 的复合图，放在 Architecture 章节。
Critical steps dashboard	00:41:10–00:42:30	用于 Infrastructure 的 bandmark 描述，配 ‘figure’ 说明。

表 4: 当前 available frames 与其 intended placement，后续若有 slides 也可在 same table 中加行。

Frame/Slide 记录要求

记录每张 figure 的 local filename、timecode、所属章节与 \protect\footnotemark，并在实战流程里把这张图的技术点、quote 与 summary 顺带整理，加速 audit 复核。

9.3 本章小结

Evidence Matrix 让我们对齐 “quote/timecode/figure” 三个维度，确保每个洞察都可追溯；当 slides 可用时，也可以在同一个 matrix 里补充，以保持实战文档的一致性。

10 未来方向与持续改进

10.1 Slides 与帧自动化

当 slides 版本公开后，优先使用 ‘magick -density 150 slides.pdf slide-

Visual asset automation 的三步骤

- slide-to-jpg: 用 magick 或 pdftoppm 批量转换 slide PDF。
- frame grab: 用 ffmpeg 从 audio/ video assets 里提取关键 timestamps。
- figure metadata: 记录每张图片的章节、timecode 与 ‘4’, 保证 audit traceable。

10.2 Audit-ready Checklist

根据 [QUALITY.md](../QUALITY.md) 的要求，1h+ 课程讲座需要 ≥ 20 页、 ≥ 10 箱。本次笔记目前 19 页，10 个箱；我们计划再补充一页（例如 ‘Evidence Matrix’ 的扩展），并确认 PDF 生成前后都用 ‘xelatex’ 双跑。下面是检查表格：

项目	目标（达标）	当前状态	下一步
页面数	$\geq 20p$	19p	补充 Evidence 或 Deployment 叙述，再跑一遍
Highlight Boxes	≥ 10	≥ 10	保持现有 box 数量，必要时再添加 practice box
关键节	每章本章小结 + 总结	已满足	持续复查
Metadata	真实日期/作者/URL	已填写	确认无 placeholder

表 5: Audit 关键指标与当前状态/行动项。

持续改进提醒

用一份 checklist 记录每次重写时要检查的维度（pages, boxes, metadata, figure/timecode），以及下一次复盘要做的事情（例如在 release 版里加入更多 slides/key frames）。

10.3 Action Timeline

为了把剩余的页面差补齐，我们制定了行动 timeline：先在 Evidence Matrix 里再写一段关于 tooling 的扩展，再补一张 figure，最后确认再跑 ‘xelatex’ 并更新 PDF page count。这个 timeline 也将作为 future reviewer 的 reference，用于检查后续修改是否保持一致。

Action	负责人	备注
补充 Evidence Matrix	记录者	增加一段关于 bandmark + checkpoint 的总结，目标增加 1 页内容。
新增 frame figure	视觉组	把关键 agent swarm 图截取出来，配 timecode 与 caption。
复核 audit checklist	质量组	确认 page count ≥ 20 , boxes ≥ 10 , 并写入 log。

表 6: Action timeline 用于确保有明确 agenda 去补充页数并满足 audit 目标。

Timeline tips

- 先写 Evidence Matrix 的新增段落，再把 table/figure 插进去，这样新内容会自然增加页面。
- 把 figure 与 timecode 配对，并用 `\protect\footnotemark` 标注，确保新的视觉证据也能复现。
- 完成后再跑一次 ‘xelatex’，用 ‘pdftinfo’ 确认页面数并更新 audit script 结果。

10.4 本章小结

未来方向包括把 slides/帧处理自动化、持续补充 evidence，以及用 checklist 手机 audit 数字，形成可复用的运营 playbook。

11 总结与延伸

11.1 关键点

- “Dive into Kimi K2.5” 把整份讲稿框在 data、algorithm、infra 三条主线里，方便我们按逻辑拆解技术报告。
- 早期就混入视觉数据、用 benchmark 驱动的合成 pipeline 是 K2.5 能在图文、代码、GUI 上打通能力的基础。
- GRM 成为统一 reward，Pretrain/SFT/RL 三段训练叠加出稳态能力，deg-and-recover 是正常的收敛轨迹。
- Agent swarm 的魅力在于可学的 Allcastrater、并行 subagent 与 critical steps 资源约束，使得多 Agent 比单 Agent 更高效。
- Infrastructure 通过 Critical Steps + Kimi code bench 进行评估，即便讲师少解释，训练曲线和 bandmark 已经证明了该体系可行。
- 实战部分展示了如何把字幕、关键帧、slides 结合成具备 summary table 与 box 层次的文档，确保 audit 通过。
- 最终，原生多模态 + Agent Swarm 让 K2.5 真正具备了 “Visual Agency” 的执行力，而不仅是视觉理解。

11.2 总结表

维度	洞见	证据
Data	早期混入视觉、定义 benchmark, 再用合成 pipeline 逐阶段扩展能力。	附录 Figure 9 中 1:9 1:1 的比例实验; 讲师列出的三步数据构建流程。
Algorithm	GRM + PARL RL 统一 reward, 多阶段训练让 open-ended 任务有稳定度; Deg & Recover 是自然波动。	GRM 知识盒与 warningbox 里的解释, PARL agent loop、critical steps 训练曲线。
Agent & Infra	Allcastrater 确保 subagent 调度、Critical Steps 限定资源, Evaluation 使用 Kimi code bench 与 bandmark。	Agent section 中 createSub-agents/assignTask 讲解 + infra section 提到评估 bandmark。
Practice	实战章节规范化素材处理、关键帧与 slides 的插图、summary table, 确保文档满足 audit 要求。	实战章节中的 knowledgebox、importantbox 以及 figure timecode 示例。

表 7: 从数据、算法到 agent/infra 的层层产业化, 体现 K2.5 原生多模态 + Agent Swarm 架构的逻辑闭环。

11.3 拓展阅读

- Kimi K2.5 | Open Visual Agentic Model for Real Work —官方模型概览与接口说明。
- Kimi K2.5 Tech Blog: Visual Agentic Intelligence —详细介绍 Visual Agency Intelligence 与数据策略。
- Kimi K2.5 —Everything you need to know (Artificial Analysis) —外部视角的 benchmark 与 agent swarm 小结。
- Kimi K2.5: A Deep Dive into China's Long-Context AI Model —比较 + 适用场景分析。
- Kimi API Platform Guide —说明视觉 agent 调用和工具链的实践方法。

11.4 本章小结

这场串讲最终告诉我们: K2.5 构建了一个原生多模态的 agentic stack, 数据、GRM 驱动的算法与 infra/agent 协调形成闭环, 赋予模型真正的 Visual Agency Intelligence。